

Bölüm 6

Bellek Yönetimi

Kapsamlı Konu Anlatımı — Türkçe

Temel kaynak: Modern Operating Systems, 5th Ed. (Tanenbaum & Bos)
Ders Notları: Yakup Genç, 2026

- Bellek Kaynağı ve Tarihsel Gelişim
- Yer Değiştirme (Relocation) Yöntemleri
- Takas (Swapping) ve Sıkıştırma
- Bellek Tahsis Algoritmaları (First/Best/Worst Fit)
- Sanal Bellek ve Sayfalama (Paging)
- Sayfa Tablosu Yapısı, TLB, Segment Tabloları
- Sayfa Değiştirme Algoritmaları (NRU, FIFO, LRU, Clock, WSClock)
- Çalışma Kümesi (Working Set) ve Thrashing
- Segmentasyon ve MULTICS/Pentium Modelleri
- Kitap Bölüm Sonu Soruları ve Çözümleri

1. Bellek Yönetimine Giriş

İşletim sistemlerinin temel görevlerinden biri bellektir (main memory). Bellek, işlemcinin doğrudan erişebildiği tek depolama birimidir; bu nedenle her çalışan program fiziksel veya sanal olarak bellekte yer kaplamak zorundadır. OS, belleği bir kaynak olarak yönetir: hangi process'in ne kadar bellek kullanacağına karar verir, process'ler arası izolasyonu sağlar ve bellek tükendiğinde taşma stratejileri (swapping, paging) uygular.

✦ Bellek Yönetiminin Temel Soruları

- Relocation: Program farklı adreslerde yüklenebilmeli mi?
- Protection: Bir process başkasının belleğine erişememeli.
- Allocation: Boş bellek en verimli şekilde nasıl dağıtılır?
- Virtual Memory: Fiziksel bellekten büyük adres uzayı nasıl sağlanır?

1.1 Bellek Düzeni — Basit Model

En basit modelde işletim sistemi altta, uygulama üstte konumlanır:

Bölge	Açıklama
Üst: Uygulama Belleği	User-space process'lerin kodu ve verisi
Alt: İşletim Sistemi	Çekirdek (kernel) kodu ve veri yapıları

Ancak çok programlı (multiprogramming) sistemlerde bu model yetmez; birden fazla process bellekte aynı anda bulunur ve yönetim çok daha karmaşık hale gelir.

2. Bellek: Tarihsel Perspektif ve Statik Overlay

Bilgisayar tarihi boyunca bellek her zaman kıt ve pahalı bir kaynak olmuştur. Dönemin teknolojisi ne olursa olsun, mevcut uygulamaların tümüne yetecek bellek hiçbir zaman olmadı. Bu baskı altında programcılar ve OS tasarımcıları yaratıcı çözümler üretmek zorunda kaldı.

2.1 Statik Overlay (Katmanlı Yükleme)

Çok eski sistemlerde programcılar bellek yönetimini kendileri yapıyordu. Farklı alt programlar (subroutine) aynı bellek alanını sırayla kullanıyordu — buna static overlay denir. OS yalnızca yardımcı kütüphane rutinleri sunuyordu; düzen ve güvenlik tamamen programcının sorumluluğundaydı.

⚠ Statik Overlay — Nasıl Çalışır?

1. Program birkaç "katman" (overlay section) olarak bölünür.
 2. Katman A çalışır → tamamlanınca disk'ten Katman B yüklenir, A'nın belleğini alır.
 3. Böylece 64 KB'lık belleğe 200 KB'lık program sığdırılabilir.
- Sorun: Hatalara açık, bakımı zor, her değişiklikte manuel güncelleme gerekir.

3. Çok Programlama ve Bellek Sorunları

Bellek kapasitesinin artmasıyla işletim sistemleri birden fazla programı aynı anda bellekte tutmaya başladı (multiprogramming). Bu yeni sorunları beraberinde getirdi:

Sorun	Açıklama	Örnek
Relocation	Program farklı adreste yüklenebilmeli	JMP 0x1000 → JMP 0x5000
Protection	Processler birbirinin belleğine erişememeli	App A, App B'yi okumamalı
Allocation	Boş bellek bloklarının verimli dağıtımı	First Fit / Best Fit

3.1 Yer Değiştirme (Relocation) Çözümleri

Üç temel yaklaşım vardır:

3.1.1 Konum Bağımsız Kod (PIC — Position-Independent Code)

Bazı kodlar doğası gereği konum bağımsızdır: mutlak adres yerine program sayacı (PC) veya register'a göreli adres kullanır. Modern Unix varyantlarındaki paylaşımlı kütüphaneler (shared libraries) bu yöntemi kullanır.

```
; Konum bağımsız assembly örneği
L R3, 16(R12) ; R12+16 adresinden yükle (relative)
B *-4         ; 4 byte geri dal (PC-relative)

; gcc ile PIC üretmek için:
gcc -fpic -shared mylib.c -o mylib.so
```

✓ PIC — Avantaj / Dezavantaj

Avantaj: Program herhangi bir adrese yüklenebilir, kopyalanabilir.

Avantaj: Shared library'ler birden fazla process tarafından paylaşılabilir.

Dezavantaj: Her mimari için uygun değil; yüklendikten sonra taşımaya yardımcı olmaz.

3.1.2 Yükleme Zamanı Yer Değiştirme (Load-Time Relocation)

Derleyici ve bağlayıcı (linker) programın 0'dan başlayacağını varsayar. Yükleyci (loader) programı gerçek adrese kopyalarken tüm mutlak adreslere başlangıç offset'ini ekler.

```
Derleyicinin varsaydığı: JUMP 0x0200
Gerçek yükleme adresi: 0x4000

Düzeltilmiş kod:      JUMP 0x4200    (0x0200 + 0x4000)

→ Her mutlak adres referansı için bu işlem tekrarlanır.
→ Relocatable entry sayısı mimariye ve derleyiciye göre değişir.
```

⚠ Load-Time Relocation Neden Sınırlı?

- Program yüklenince adresi sabitlenir — daha sonra başka yere taşınamaz.
- Çok sayıda adres referansı varsa yükleme yavaşlar.
- Yükleme tamamlandıktan sonra PIC gibi çalışmaz.

3.1.3 Donanım Destekli Yer Değiştirme — Base/Limit Register

OS, her process için bir base (taban) ve limit (sınır) register'ı ayarlar. Tüm bellek referanslarına otomatik olarak base eklenir; limit'i aşan erişimler donanım tuzağı (trap) tetikler.

Fiziksel Adres = Sanal Adres + Base Register

Örnek:

Base = 0x10000 (OS process'i 64 KB'a yükledi)
Limit = 0x0FFFF (process 64 KB büyüklüğünde)
Sanal = 0x00500
Fiziksel = 0x10500

Eğer Sanal > Limit → TRAP (Segmentation Fault)

✖ Base/Limit Sorunları

- Her bellek referansında bir toplama işlemi → performans maliyeti.
- Programları başka yere kopyalamak pahalıdır (tüm bellek kopyalanır).
- Esneklik yetersiz: tek segment, büyük boşluklar kullanılamaz.

4. Takas (Swapping)

Swapping, bellekte yer açmak için bir process'in tüm içeriğini diske yazıp daha sonra geri yükleme tekniğidir. Bloklanmış (blocked) bir process bellekten çıkarılır, CPU serbest kalır ve diske ayrılan "swap alanı"na yazılır.

Adım	İşlem	Açıklama
1	Disk'e Yazma	Bloklanmış process'in tüm belleği diske kopyalanır
2	Bellek Serbest	O process'in kapladığı bellek alanı serbest kalır
3	Yeni Process	Başka bir process bu alana yüklenebilir
4	Geri Yükleme	Bloke process hazır olunca diskten geri yüklenir
5	Farklı Adres?	Yükleme farklı adrese yapılabilir → relocation gerekir

⚠ Compaction (Bellek Sıkıştırma)

Zamanla bellek parçalanır (fragmentation). Swap kullanarak processler diske çıkarılıp geri alınırken bellek yeniden düzenlenebilir. Ancak compaction pahalıdır: tüm içerik kopyalanır, bellek bandwidthini tüketir.

✖ Swap'ın Gizli Maliyeti

"Disk I/O CPU'yu serbest bırakır" gibi görünse de disk yazma/okuma işlemleri bellek veri yolunu (memory bandwidth) meşgul eder. Sistem CPU değil memory bus açısından darboğaza girer.

5. Bellek Tahsis Algoritmaları

OS, hangi boş bellek bloğunun bir process'e verileceğine karar vermek zorundadır. Bu karar için birkaç klasik algoritma vardır. Boş bloklar genellikle bağlı liste (linked list) ile takip edilir.

5.1 First Fit (İlk Uygun)

Listeyi baştan tara; isteği karşılayacak büyüklükteki ilk bloğu bul ve tahsis et. Kalan kısım listeye geri döner.

Boş bloklar: [10MB] [4MB] [20MB] [18MB] [7MB]

İstek: 12MB

→ İlk taranan: 10MB (yetersiz)

→ 4MB (yetersiz)

→ 20MB ✓ → 20MB'dan 12MB al, 8MB geri ver

Sonuç: [10MB] [4MB] [8MB] [18MB] [7MB]

✦ First Fit — Özellikler

- Hızlıdır: tüm listeyi taramak zorunda değil.
- Listenin başında küçük parçalar birikir → external fragmentation.
- Uzun vadede listeyi adrese göre sıralı tutmak coalescence'i kolaylaştırır.

5.2 Best Fit (En Uygun)

Tüm listeyi tara; isteğe en yakın boyuttaki bloğu seç. Amacı: büyük blokları mümkün olduğunca korumak.

Boş bloklar: [10MB] [4MB] [20MB] [18MB] [7MB]

İstek: 12MB → En yakın uygun: 18MB
18MB'dan 12MB al, 6MB geri ver.

Sorun: Küçük, kullanışsız parçalar oluşur → fragmentation kötüleşir!

⚠ Best Fit — Gerçekte Daha Kötü

Sezgisel olarak "en verimli" gibi görünse de Best Fit, First Fit'ten daha iyi değildir. Küçük ve işe yaramaz parçalar bırakır (fragmentation artar). Simülasyonlar First Fit $\approx$ Best Fit > Worst Fit sonucunu göstermiştir.

5.3 Worst Fit (En Büyük Blok)

Her seferinde en büyük bloğu seç ve böl. Mantık: büyük bloktan parça alınca geri kalan hâlâ kullanışlı olabilir. Ancak pratikte en kötü performansı verir.

5.4 Algoritma Karşılaştırması

Algoritma	Hız	Fragmentation	Pratikte Kullanım
First Fit	Hızlı	Orta (liste başı)	Sık kullanılır
Best Fit	Yavaş (tam tarama)	Yüksek (küçük parçalar)	Nadir
Worst Fit	Yavaş	En yüksek	Neredeyse hiç
Next Fit	Hızlı	First Fit benzeri	Bazen

✖ 1/3 Kuralı (Fifty-Percent Rule)

First Fit için yapılan simülasyonlar göstermiştir: belleğin yaklaşık 1/3'ü external fragmentation nedeniyle kullanılamaz olabilir. Bu, klasik bellek tahsisinin temel motivasyonunun neden sanal belleğe yöneldiğini açıklar.

6. Sanal Bellek (Virtual Memory)

Sanal bellek, programların gördüğü adres uzayı (virtual address space) ile fiziksel RAM arasına bir soyutlama katmanı ekler. Bu katman sayesinde:

✓ Sanal Belleğin Çözdüğü Sorunlar

- ✓ Relocation: Her process aynı sanal adresten başlayabilir (ör: 0x0000).
- ✓ Protection: Fiziksel bellek haritası ayrı olduğu için process'ler birbirini göremez.
- ✓ Allocation: Fiziksel bellekteki boşluklar sorun değil; sanal adresler bitişik görünür.
- ✓ Büyük Adres Uzaı: Fiziksel RAM'den büyük programlar çalıştırılabilir.
- ✓ Paylaşım: Ortak sayfalar (shared libraries) birden fazla process'e map edilebilir.

6.1 Temel Kavramlar

Kavram	Açıklama
Virtual Address	Programın gördüğü adres (process'e özel)
Physical Address	RAM'deki gerçek adres
Page	Sanal adres uzayının sabit boyutlu birimi (genellikle 4 KB)
Page Frame	Fiziksel RAM'deki sayfa boyutlu alan
Page Table	Sanal → Fiziksel sayfa eşlemesini tutan tablo
MMU	Memory Management Unit — adresi donanımda çeviren birim
TLB	Translation Lookaside Buffer — sayfa tablosu önbelleği

7. Sayfalama (Paging) ve MMU

Sayfalama, sanal adres uzayını eşit boyutlu sayfalara (page), fiziksel belleği de aynı boyutlu page frame'lere böler. MMU her sanal adresi çevirirken sayfa tablosuna bakar.

7.1 Adres Dönüşümü

Sanal adres iki parçadan oluşur: üst bitler sayfa numarası (V), alt bitler ise sayfa içi ofsettir (O).

Sanal Adres $A = \langle V, 0 \rangle$

V = Sanal Sayfa Numarası (virtual page number)
 0 = Offset (sayfa içi konum)

MMU dönüşümü:
 $A \rightarrow \langle M[V], 0 \rangle$

$M[V]$ = Fiziksel sayfa çerçevesi (page frame)

Örnek (4 KB sayfa boyutu):
Sanal adres: 8196 = 0x2004
 $V = 0x2004 \gg 12 = 2$ (sayfa numarası)
 $0 = 0x2004 \& 0xFFF = 4$ (offset)
 $M[2] = 110$ (page frame 110)
Fiziksel = $110 * 4096 + 4 = 450564$

7.2 Sayfa Tablosu Örneği

Aşağıdaki tabloda 8 sayfalık bir process'in sanal $\rightarrow$ fiziksel eşlemesi gösterilmektedir. Bazı sayfalar bellekte değil (valid bit = i = invalid), yalnızca diskte bulunur.

Sanal Sayfa	Frame (Fiziksel)	Valid/Invalid	Anlamı
0 (A)	4	valid (v)	RAM'de, frame 4
1 (B)	-	invalid (i)	Diskte / yüklenmemiş
2 (C)	6	valid (v)	RAM'de, frame 6
3 (D)	-	invalid (i)	Diskte
4 (E)	-	invalid (i)	Diskte
5 (F)	-	invalid (i)	Diskte
6 (G)	-	invalid (i)	Diskte
7 (H)	-	invalid (i)	Diskte

7.3 Sayfa Tablosu Girdisi (Page Table Entry — PTE) Yapısı

Bir PTE tipik olarak şu alanları içerir:

Alan	Bit Boyutu	Açıklama
Page Frame Number	Büyük kısım	Fiziksel frame numarası
Present/Absent (V)	1 bit	1=RAM'de, 0=diskte veya geçersiz
Protection (prot)	2-3 bit	Read / Write / Execute izinleri
Modified (M)	1 bit	"Dirty bit" — sayfa değiştirildi mi?
Referenced (R)	1 bit	Son clock tick'te erişildi mi?
Caching Disabled	1 bit	I/O portları için önbelleklemeyi kapat

8. TLB ve Çok Seviyeli Sayfa Tabloları

8.1 Sayfa Tablosunun RAM'de Tutulması — Sorun

Sayfa tablosunu RAM'de tutmak iki sorun yaratır:

⚠ Sayfa Tablosunun Maliyeti

- Hız: Her bellek erişiminde ekstra bir RAM okuma → 2x yavaşlama.
- Boyut: 32-bit sistem, 4 KB sayfa → 1 milyon PTE → 4 MB/process (64-bit'te astronomik boyuta ulaşır).
- Çözüm: TLB (Translation Lookaside Buffer) ve çok seviyeli tablolar.

8.2 TLB — Translation Lookaside Buffer

TLB, en son kullanılan sayfa çevirilerini tutan küçük ve son derece hızlı asosyatif bellek (associative memory) birimidir. Sanal sayfa numarası → fiziksel frame eşlemelerini önbelleğe alır.

TLB Miss Oranı Düşük Olduğu Sürece:

$$T_{eff} = h * T_{TLB} + (1-h) * (T_{TLB} + T_{RAM})$$

h = TLB hit oranı (genellikle 0.95 - 0.99)

T_{TLB} = TLB erişim süresi (~1-2 ns)

T_{RAM} = RAM erişim süresi (~100 ns)

Örnek: $h=0.99$, $T_{TLB}=1ns$, $T_{RAM}=100ns$

$$T_{eff} = 0.99*1 + 0.01*(1+100) = 0.99 + 1.01 = 2 \text{ ns}$$

(Sayfa tablosu RAM'de olsa: 101 ns)

Durum	TLB İçin Yapılan	Ek RAM Erişimi
TLB Hit	Direkt frame adresi alınır	Yok
TLB Miss	RAM'deki sayfa tablosuna gidilir, TLB güncellenir	1 ek erişim
TLB Miss + Page Fault	Disk'ten sayfa yüklenir, tablo güncellenir	Disk I/O (~ms)

✦ TLB'nin Arkasındaki Prensip

TLB, referans yerelliği (locality of reference) prensibine dayanır: bir program kısa süre içinde genellikle az sayıda farklı sayfaya erişir. Bu nedenle küçük bir TLB bile yüksek hit oranı sağlar. RISC sistemlerde TLB yazılım (OS) tarafından doldurulur; x86'da donanım yönetir.

8.3 Segment (İki Seviyeli) Sayfa Tabloları

Modern 32/64-bit sistemlerde sanal adres alanı çok büyüktür (ör: 4 GB veya daha fazla). Tek düzey sayfa tablosu çok yer kaplar. Çözüm: adresin bölümlere ayrılması.

32-bit adres, 2 seviyeli sayfa tablosu:

31 22 | 21 12 | 11 0
[PT1: 10 bit] [PT2: 10 bit] [Offset: 12 bit]

Dönüşüm:

1. PT1 → 1. seviye sayfa tablosunun indisi
Bu entry 2. seviye tablonun adresini verir.
2. PT2 → 2. seviye tablonun indisi
Bu entry fiziksel frame numarasını verir.
3. Fiziksel adres = Frame * 4096 + Offset

Avantaj: Kullanılmayan adres alanları için sayfa tablosu oluşturulmaz → büyük tasarruf!

8.4 Tersine Sayfa Tablosu (Inverted Page Table)

Geleneksel sayfa tablosu her sanal sayfanın frame'ini tutar — sanal adres uzayıyla orantılı büyür. Tersine tabloda ise her fiziksel frame için bir giriş vardır; hangi process'in hangi sanal sayfasına karşılık geldiği hash ile bulunur.

Özellik	Geleneksel Tablo	Tersine Tablo
Boyut	Sanal adres uzayıyla orantılı (büyük)	Fiziksel RAM ile orantılı (küçük)
Arama	O(1) — direk indeks	Hash + bağlı liste — O(1) ortalama
Paylaşım	Kolay	Zor
Kullanım	x86, MIPS	PowerPC, IA-64

9. Sayfa Hataları (Page Faults)

Bir process geçersiz (invalid) bir sayfaya eriştiğinde donanım bir sayfa hatası (page fault) kesmesi üretir. Çekirdek bu isteği işler:

Sayfa Hatası İşleme Adımları:

1. MMU: geçersiz PTE → hardware trap → çekirdeğe geç
2. Çekirdek: hangi sanal adres → neden? kontrol et
 - a) Program hatası mı? (null pointer, sınır dışı erişim)
→ Process'e sinyal gönder (SIGSEGV)
 - b) Sayfa diskte mi? (valid ama not-present)
→ Disk I/O planla: boş frame bul (veya bir sayfayı evict et)
→ Sayfayı diskten RAM'e yükle
→ PTE'yi güncelle (valid=1, frame=yeni frame)
→ Process'i devam ettir (aynı komutu yeniden çalıştır)

⚠ Page Fault Donanım Gereksinimi

Sayfa hatasında kesilen komutun yeniden çalıştırılabilmesi (restartable) gerekir. Komut çok operand içeriyorsa ve operandlardan biri sayfa sınırını aşıyorsa (cross-page-boundary), iki sayfa hatası olabilir — her iki sayfa da yüklenmeli!

10. Sayfa Değiştirme Algoritmaları

Fiziksel bellek dolduğunda yeni bir sayfa yüklemek için mevcut sayfalardan biri evict edilmeli. Hangi sayfanın seçileceği büyük performans farkı yaratır. Her algoritmanın temel aracı: R (Referenced) ve M (Modified) bitleridir.

✦ OS'un Elindeki Araçlar

- R biti (Referenced): Sayfa yakın zamanda kullanıldı mı?
- M biti (Modified/Dirty): Sayfa değiştirildi mi? Diske yazılması gerekir.
- Clock kesmesi: Periyodik olarak R bitleri sıfırlanır.
- Sayfa hatası kesmesi: Hangi adres erişildi bilgisi.
- Uygulama ipuçları: madvise() gibi system call'lar.

10.1 Optimal Algoritma

Teorik ideal: en uzun süre kullanılmayacak sayfayı değiştir. Uygulanamaz (geleceği bilemeyiz), ama benchmark için kullanılır.

10.2 NRU — Not Recently Used

Sayfaları R ve M bitine göre 4 sınıfa ayırır ve en düşük sınıftan rastgele biri seçilir. Çok basit; öğretim amaçlı kullanılır.

Sınıf 0: R=0, M=0 → Ne kullanıldı ne değiştirildi (EN İYİ aday)
Sınıf 1: R=0, M=1 → Değiştirildi ama son zamanda erişilmedi
Sınıf 2: R=1, M=0 → Son zamanda erişildi, değişmedi
Sınıf 3: R=1, M=1 → Hem erişildi hem değiştirildi (EN KÖTÜ aday)

Her clock tick'te: R bitleri sıfırlanır
Sayfa hatası olunca: en düşük sınıftan rastgele seç

⚠ Neden M Sıfırlanmaz?

R biti clock'ta sıfırlanır çünkü "son zamanda kullanıldı mı?" sorusunu yanıtlar. M biti sıfırlanamaz çünkü sayfa diske senkronize edilmeden M=0 yapmak veri kaybına yol açar. M ancak sayfanın diske yazılmasıyla sıfırlanabilir.

10.3 FIFO — İlk Giren İlk Çıkar

Sayfalar yüklenme zamanına göre sıralanır; en eskisi atılır. Sorun: en eski sayfa hâlâ aktif kullanımda olabilir.

✖ Belady'nin Anomalisi (Belady's Anomaly)

FIFO'da daha fazla frame DAHA FAZLA sayfa hatasına yol açabilir! Örnek referans dizisi: 3 2 1 0 3 2 4 3 2 1 0 4
→ 3 frame ile 9 sayfa hatası
→ 4 frame ile 10 sayfa hatası (!)
Bu anomali yalnızca stack algoritması olmayan yöntemlerde (FIFO gibi) görülür.

10.4 Second Chance FIFO — İkinci Şans

FIFO'yu geliştirir: en eski sayfanın R biti kontrol edilir. R=1 ise bit sıfırlanır ve sayfa listenin sonuna taşınır (ikinci şans verilir). R=0 ise sayfa hemen evict edilir.

FIFO listesi (en eski → en yeni): A(0) → B(3) → C(7) → D(8) → ...

Sayfa hatası → A incelenir:

A.R = 1? → R=0 yap, A'yı listenin SONUNA taşı → B'ye geç

B.R = 0? → B evict edilir → yeni sayfa B'nin yerine yüklenir

10.5 Clock Algoritması

Second Chance FIFO'nun verimli implementasyonu. Sayfalar dairesel bir tampon (circular buffer) üzerinde tutulur; bir "saat ibresi" (clock hand) döner. İbre R=1 gördüğünde R=0 yapar ve ilerler; R=0 görünce o sayfayı değiştirir.

```
WHILE (true):
    sayfa = clock_hand'in gösterdiği sayfa
    IF sayfa.R == 1:
        sayfa.R = 0           # ikinci şans ver
        clock_hand ilerle
    ELSE: # R == 0
        bu sayfayı evict et
        yeni sayfayı buraya yükle
        clock_hand ilerle
    BREAK
```

Mantık aynı fakat Clock, Second Chance'in verimli versiyonudur: sayfa tablosu girdisini fiziksel olarak taşımak yerine yalnızca ibrenin işaret ettiği konum değişir. Linux kernel'i clock tabanlı bir algoritma kullanır.

En son kullanılanı en uzak zaman önce kullanılabilecek sıralar; en eskiyi evict eder. Referans yerelliğine göre optimal'a en yakın pratik yaklaşımdır. Ancak tam implementasyonu pahalıdır.

```
Donanım Destekli LRU –  $n \times n$  Matris Yöntemi:
  n sayfa için  $n \times n$  bit matrisi M, başta sıfır.
  Sayfa k erişilince: satır k  $\rightarrow$  tüm bitleri 1 yap
                        sütun k  $\rightarrow$  tüm bitleri 0 yap
  En düşük ikili değerli satır  $\rightarrow$  LRU sayfası
```

LRU'nun yazılım simülasyonu. Her sayfa için bir sayaç tutulur. Sorun: NFU hiç unutmaz — eski yoğun kullanım hâlâ sayacı yüksek tutar. Çözüm: Aging — her clock tick'te sayacı sağa kaydır (ikiye böl), sonra R bitini en yüksek bite ekle.

NFU With Aging – Her clock tick için:

```
FOR her sayfa p:
    p.counter >>= 1          # sağa kaydır (÷2)
    p.counter |= (p.R << 7)  # R bitini en üst bite ekle
    p.R = 0                  # R bitini sıfırla
```

Sayfa hatasında: en küçük counter → evict

Örnek (8-bit counter, 3 tick sonra):

Sayfa A: R: 1,0,1 → 10100000 → 10110000 → 01011000...

Sayfa B: R: 0,0,0 → 00000000 (evict adayı!)

✓ Aging – Neden İyi?

- Son kullanımlar daha ağır, eskiler daha hafif (exponential decay).
- Geçmiş tamamen unutulmaz; yakın geçmiş ağırlıklı.
- $O(n)$ zaman karmaşıklığı, her clock tick'te — makul maliyet.
- Pratikte iyi sonuçlar verir; Linux'ta benzeri mekanizmalar kullanılır.

10.8 Algoritma Karşılaştırma Tablosu

Algoritma	Yorum
Optimal	Gerçekleştirilemez; benchmark için referans
NRU	Çok kaba LRU yaklaşımı; öğretim amaçlı
FIFO	Önemli sayfaları atabilir; Belady anomalisi riski
Second Chance	FIFO'nun büyük iyileştirmesi
Clock	Second Chance'in verimli implementasyonu; gerçekçi
LRU	Mükemmel ama tam implementasyonu pahalı
NFU	Kaba LRU yaklaşımı; geçmişi unutmaz
Aging (NFU+)	NFU'nun iyileştirilmiş; iyi pratik seçim
Working Set	Pahalı ama güçlü; thrashing önler
WSClock	Working Set + Clock; verimli ve etkili

11. Çalışma Kümesi (Working Set) ve Thrashing

11.1 Thrashing — Çırpınma

Bir process'e yetersiz frame tahsis edildiğinde, işlemci zamanının büyük çoğunluğunu sayfaları diske yazıp geri okumaya harcar. Bu duruma thrashing (çırpınma) denir.

✖ Thrashing Belirtileri

- CPU kullanımı düşer (process sürekli I/O bekler).
- Sayfa hata sayısı aniden yükselir.
- Sistem çok yavaşlar — eklenen process'ler durumu kötüleştirir.
- Disk erişim ışığı sürekli yanar.

11.2 Yerellik Prensibi (Locality Principle)

Programlar bir anda tüm sayfalarını kullanmaz. Belirli bir sürede yalnızca küçük bir sayfa kümesine (locality) erişir. Yürütme ilerledikçe locality değişir ama ani geçişler nadirdir.

11.3 Working Set (Çalışma Kümesi)

$w(k, t) = t$ anında en son k bellek referansında erişilen farklı sayfa kümesi. k büyüdükçe working set monoton artar ve program boyutuna asimptotik yaklaşır.

Referans dizisi: P1 P2 P3 P1 P4 P1 P4 P3 P5 P6 P1

$k=4$ ile working set penceresi:

...P1 P4 P1 P4 → $W = \{P1, P4\}$

...P4 P3 P5 P6 → $W = \{P3, P4, P5, P6\}$

...P3 P5 P6 P1 → $W = \{P1, P3, P5, P6\}$

Kullanım: Programı çalıştırmadan önce working set'ini RAM'e yükle.

Working set RAM'de → sayfa hatası yok.

Working set RAM'de değil → thrashing.

11.4 Working Set Algoritması — Yaklaşım

k son referansı izlemek pahalıdır. Bunun yerine son τ saniyede erişilen sayfalar izlenir. Her sayfa için time of last use kaydedilir.

Working Set Sayfa Değiştirme:

Her sayfa için: [time_of_last_use, R_bit]

Sayfa hatası olduğunda tüm sayfaları tara:

IF $R == 1$:

time_of_last_use = current_virtual_time

$R = 0$

ELIF $R == 0$ AND (current_time - time_of_last_use) > τ :

Bu sayfa working set dışı → evict et!

ELIF $R == 0$ AND age <= τ :

En küçük time_of_last_use'lu sayfayı hatırla

Hiç evict edilecek yoksa: en küçük time'lı sayfayı al

11.5 WSClock Algoritması

Working Set prensibi + Clock veri yapısı = WSClock. Dairesel bağlı liste kullanılır. Her sayfa hatasında clock hand ilerler:

WSClock – Karar Tablosu:

```
R=1: R=0 yap, ibreyi ilerlet (working set'te, dokunma)
R=0 + age >  $\tau$  + M=0: Hemen evict et (temiz, eski)
R=0 + age >  $\tau$  + M=1: Diske yazma planla, ibreyi ilerlet
R=0 + age  $\leq \tau$ : Working set'te, ibreyi ilerlet
```

Tam tur sonunda:

- Yazma planlanmışsa: bitene kadar bekle
- Yazma yoksa: rastgele temiz sayfayı al

✓ WSClock — Neden İyi?

- Working set kavramını uygular → thrashing önler.
- Clock yapısı sayesinde her page fault'ta $O(n)$ tam tarama gerekmez.
- Dirty sayfalar için I/O önceden planlanır (anticipatory paging).
- Gerçek OS'larda kullanılır; Linux'taki page reclaim benzer prensiplere dayanır.

11.6 Page Fault Frekansı (PFF) ile Kontrol

Thrashing'i dinamik olarak tespit etmek için her process'in sayfa hata frekansı izlenir.

```
PFF > Üst Sınır → Process'e daha fazla frame ver
PFF < Alt Sınır → Process'ten frame geri al
```

Eğer vermek için yeterli frame yoksa:

- Bir process'i swap'a al, framelerini dağıt
- Daha sonra swap'tan geri getir

12. Tasarım Konuları

12.1 Yerel vs. Global Tahsis Politikası

Sayfa değiştirmede kimin frame'ine dokunulacağı kritik bir tasarım kararıdır:

Politika	Kimin Frame'i Alınır?	Avantaj	Dezavantaj
Yerel (Local)	Yalnızca hata yapan process'in kendi frame'i	Process'ler birbirini etkilemez	Working set büyüdüğünde thrashing riski
Global	Tüm process'lerin frame havuzu	Daha iyi sistem geneli throughput	Process'in performansı diğerlerine bağlı

✓ Sonuç

Global tahsis genellikle daha iyi sistem geneli performans sağlar ve yaygın kullanılır. Ancak global politikada her process'e ne kadar frame verileceği ayrıca belirlenmeli. PFF tabanlı dinamik tahsis bu sorunu çözer.

12.2 Sayfa Boyutu (Page Size)

Sayfa boyutu p , bellek israfı ve sayfa tablosu boyutu arasındaki dengeyi belirler. Ortalama process boyutu s , sayfa tablosu entry boyutu e olsun:

```
Toplam overhead:  o = (s/p)*e  +  p/2
                   ^^^^^^^^^^  ^^^^^
                   Sayfa tablosu Internal fragmentation
                   bellek israfı  (ortalama yarım sayfa boşa gider)
```

$do/dp = 0$ koşulundan optimal sayfa boyutu:
 $p_{opt} = \sqrt{2 * s * e}$

Örnek: $s=1MB$, $e=8B \rightarrow p_{opt} = \sqrt{2*1M*8} \approx 4 \text{ KB}$ ✓

12.3 Paylaşılan Sayfalar ve Kütüphaneler

Birden fazla process aynı kodu çalıştırıyorsa (ör: bash, libc), kod sayfaları fiziksel bellekte tek kopya tutulabilir; her process kendi sayfa tablosundan aynı frame'lere işaret eder.

✦ Copy-on-Write (CoW)

`fork()` sistem çağrısında parent ve child aynı sayfalara işaret eder, tüm sayfalar read-only işaretlenir. Bir process yazmaya çalışınca koruma hatası $\rightarrow$ kernel sayfayı kopyalar ve yalnızca o process'e yazar. Böylece gereksiz kopyalama önlenir.

12.4 Kilitleme (Wiring/Locking)

Gerçek zamanlı (real-time) sistemler veya bazı I/O işlemleri için bazı sayfaların kesinlikle RAM'de kalması gerekir. Bu sayfalara wired veya locked denir. Çok sayıda kilit sistem performansını düşürür — bu nedenle root yetkisi veya kota gerektirir.

13. Segmentasyon (Segmentation)

Sayfalama doğrusal bir adres uzayı sunar — program kod, veri, yığın gibi mantıksal olarak farklı bölümleri tek alanda birleştirmek zorunda kalır. Segmentasyon bu bölümlere ayrı, bağımsız adres uzayları tanır.

13.1 Neden Segmentasyon?

✓ Segmentasyonun Avantajları

- Kod (text) segmenti: salt okunur, paylaşılabilir.
- Veri (data) segmenti: okuma-yazma.
- Stack segmenti: dinamik büyüme.
- Her segment bağımsız büyüyüp küçülebilir — segment sınırları karışmaz.
- Paylaşılan kütüphane tek segment olarak her processe aynı sayfa tablosuyla eklenir.
- Her segmente bağımsız erişim hakları (protection bits) atanabilir.

13.2 Segmentasyon Sorunları

✗ Segmentasyonun Dezavantajları

- Her segment için maksimum boyut sınırı vardır (ör: Intel 286'da 64 KB).
- External fragmentation: segmentler değişken boyutlu → compaction gerekir.
- Saf segmentasyon günümüz OS'larında nadir; genellikle sayfalama ile birleştirilir.

13.3 MULTICS — Segmentasyon + Sayfalama

MULTICS (1960'lar) her ikisini birleştirdi: sanal adres = segment numarası + sayfa numarası + offset. Descriptor segment tüm segment tablolarını işaret eder.

MULTICS 34-bit Sanal Adres:

[Segment: 18 bit] | [Sayfa: 6 bit] | [Offset: 10 bit]

Dönüşüm:

1. Segment numarası → Descriptor segment'ten segment descriptor'ı al
2. Descriptor → Sayfa tablosunun adresini ver
3. Sayfa numarası → Sayfa tablosundan fiziksel frame al
4. Fiziksel adres = Frame * page_size + Offset

Sayfa hatası veya segment hatası → kernel devreye girer

13.4 Pentium (x86) Segmentasyonu

Intel x86 mimarisi hem segmentasyon hem sayfalama destekler. 6 segment register vardır (CS, DS, SS + 3 yedek). Her segment descriptor 32-bit base + limit içerir. İki seviyeli sayfa tablosuyla birleşince 3 ek bellek erişimi gerekebilir — bu yüzden TLB kritiktir.

Pentium Adres Dönüşümü:

Segment Register (Selector) → GDT/LDT'den Descriptor

Descriptor: Base + Limit + Protection

Linear Adres = Base + Offset (32-bit)

Linear Adres → Sayfa Tablosu (2 seviyeli)

[Dir: 10 bit] [Page: 10 bit] [Offset: 12 bit]

→ Fiziksel Adres

Toplam: 3 potansiyel RAM erişimi → TLB olmadan çok yavaş!

14. Quiz — Açık Uçlu Sınav Soruları

★ Quiz Hakkında

Aşağıdaki sorular sınav formatındadır. Önce kendiniz çözmeyi deneyin, sonra cevap bölümüne bakın. Sorular ders konularının tamamını kapsar.

S1. TLB (Translation Lookaside Buffer) nedir ve neden gereklidir? TLB hit oranı $h=0.95$, TLB erişim süresi 2 ns, RAM erişim süresi 100 ns ise efektif adres çevirme süresini hesaplayınız.

Cevap:

TLB, en son kullanılan sayfa tablosu girdilerini (V→P eşlemelerini) tutan küçük ve çok hızlı bir asosyatif önbellektir. Sayfa tablosu RAM'de tutulunca her bellek erişimi için 1 ek RAM erişimi gerektirdi; TLB bu maliyeti ortadan kaldırır.

Efektif süre hesabı:

$$\begin{aligned} T_{\text{eff}} &= h \times T_{\text{TLB}} + (1-h) \times (T_{\text{TLB}} + T_{\text{RAM}}) \\ &= 0.95 \times 2 + 0.05 \times (2 + 100) \\ &= 1.90 + 5.10 \\ &= 7.0 \text{ ns} \end{aligned}$$

TLB olmasa: her erişim 102 ns. TLB ile ~ 7 ns $\rightarrow \sim 14.5x$ hızlanma.

S2. FIFO sayfa değiştirme algoritmasında Belady Anomalisi nedir? 3 frame ile referans dizisi 3 2 1 0 3 2 4 3 2 1 0 4 için kaç sayfa hatası oluşur?

Cevap:

Belady Anomalisi: FIFO'da frame sayısı artırıldığında sayfa hatası sayısı artabilir — beklenenin tam tersi bir davranış.

3 frame, referans: 3 2 1 0 3 2 4 3 2 1 0 4

Frame durumu (FIFO, en eski solda):

3: [3] F

2: [3,2] F

1: [3,2,1] F

0: [2,1,0] F (3 atıldı)

3: [1,0,3] F

2: [0,3,2] F

4: [3,2,4] F (0 atıldı)

3: HIT

2: HIT

1: [2,4,1] F (3 atıldı)

0: [4,1,0] F

4: HIT

→ 9 sayfa hatası (F işaretli)

4 frame ile aynı dizi → 10 sayfa hatası (anomali!)

S3. NRU algoritması 4 sınıf tanımlar. Hangi process daha önce terminate edilse bile, NRU'nun R bitini neden M bitinden farklı ele aldığını açıklayınız.

Cevap:

R (Referenced) biti: Sayfanın yakın zamanda kullanılıp kullanılmadığını gösterir. Her clock kesmesinde R=0 yapılır çünkü biz "SON ZAMANDA kullanıldı mı?" sorusunu yanıtlamak istiyoruz. Sıfırlamadan sonra yeniden erişilirse R=1 olur.

M (Modified/Dirty) biti: Sayfanın diskteki kopyasından farklı olduğunu gösterir. M=1 olan bir sayfa evict edilmeden önce DISKE YAZILMALIDIR. Diskteki kopya güncel olmadan M=0 yaparsak veri KAYBI yaşarız. Bu yüzden M bitleri clock'ta sıfırlanmaz; yalnızca sayfa diske yazıldıktan sonra M=0 yapılabilir.

S4. Working set $w(k,t)$ nedir? Bir process'in working set'inin RAM'de tutulmaması durumunda ne olur ve bu durumu nasıl tespit edebilirsiniz?

Cevap:

Working set $w(k,t)$: t anında en son k bellek referansında erişilen farklı sayfaların kümesidir. k büyüdükçe $w(k,t)$ monoton artar (asla azalmaz) ve programın toplam sayfa sayısına asimptotik yaklaşır.

Working set RAM'de olmazsa:

- Process sürekli sayfa hatası üretir.
- Her hata disk I/O gerektirir (~ms mertebesinde).
- CPU bu süre boyunca boşta kalır veya başka processler çalışır.
- Bu kısır döngüye THRASHING denir.

Tespit yöntemi:

- Page Fault Frequency (PFF) izlenir.
- $PFF > \text{üst eşik}$ → thrashing, daha fazla frame ver.
- Yeterli frame yoksa → bir process'i swap'a al.

S5. İki seviyeli sayfa tablosunun tek seviyeli tabloya göre avantajı nedir? 32-bit adres, 4 KB sayfa, 4 byte PTE için tek seviyeli tablonun boyutunu hesaplayın.

Cevap:

Tek seviyeli tablo boyutu:

Sayfa sayısı = $2^{32} / 2^{12} = 2^{20} = 1.048.576$ sayfa

Tablo boyutu = $1.048.576 \times 4 \text{ byte} = 4 \text{ MB}$ (her process için!)

Sorun: Her process 4 MB sayfa tablosu taşır, büyük çoğunluğu kullanılmaz.

İki seviyeli çözüm:

Adres: [10 bit | 10 bit | 12 bit]

1. seviye: $1024 \text{ giriş} \times 4 \text{ byte} = 4 \text{ KB}$ (tek sayfa — her zaman RAM'de)
2. seviye: Yalnızca kullanılan adres aralıkları için tablo üretilir.

Boş bölgeler: 1. seviye girdisi NULL → 2. seviye tablo YOK.

Avantaj: Seyrek adres uzaylarında (stack + heap + kod) belleğin büyük kısmı boşsa, sadece birkaç 2. seviye tablo oluşturulur → büyük tasarruf.

15. Kitap Bölüm Sonu Soruları ve Çözümleri

✦ Kaynak

Modern Operating Systems, 5th Ed. — Tanenbaum & Bos, Bölüm 3: Memory Management.

Seçilen sorular ders konularıyla doğrudan ilgilidir.

Soru 3 (İng): Consider a swapping system in which memory consists of the following hole sizes in memory order: 10 MB, 4 MB, 20 MB, 18 MB, 7 MB, 9 MB, 12 MB, and 15 MB. Which hole is taken for successive segment requests of (a) 12 MB (b) 10 MB (c) 9 MB for first fit? Now repeat the question for best fit, worst fit, and next fit.

Türkçe: Soru 3: Sırasıyla 12 MB, 10 MB, 9 MB istekleri için First Fit, Best Fit, Worst Fit ve Next Fit kullanarak hangi boşluğun seçileceğini bulunuz.

Çözüm:

Boşluklar (sırayla): [10] [4] [20] [18] [7] [9] [12] [15] (MB)

First Fit:

12 MB → İlk yeterli: 20 MB (3. boşluk) → 20MB'dan 12 al, 8 bırak

Boşluklar: [10] [4] [8] [18] [7] [9] [12] [15]

10 MB → İlk yeterli: 10 MB (1. boşluk, tam sığar)

Boşluklar: [4] [8] [18] [7] [9] [12] [15]

9 MB → İlk yeterli: 18 MB (kalan) → 18'den 9 al, 9 bırak

Best Fit:

12 MB → En yakın: 12 MB (7. boşluk, tam sığar)

10 MB → En yakın: 10 MB (1. boşluk)

9 MB → En yakın: 9 MB (6. boşluk)

Worst Fit:

12 MB → En büyük: 20 MB (3. boşluk) → 8 MB kalır

10 MB → En büyük: 18 MB (4. boşluk) → 8 MB kalır

9 MB → En büyük: 15 MB (8. boşluk) → 6 MB kalır

Next Fit: (son tahsisin yapıldığı yerden devam eder)

12 MB → Baştan: 20 MB (3. boşluk) → son konum=3

10 MB → 3. konum sonrası: 18 MB (4. boşluk) → son konum=4

9 MB → 4. konum sonrası: 9 MB (6. boşluk)

Soru 7 (İng): For each of the following decimal virtual addresses, compute the virtual page number and offset for a 4-KB page and for an 8-KB page: 20000, 32768, 60000.

Türkçe: Soru 7: Aşağıdaki ondalık sanal adresler için 4 KB'lık ve 8 KB'lık sayfa boyutlarında sanal sayfa numarası ve offseti hesaplayınız: 20000, 32768, 60000.

Çözüm:

4 KB (4096 byte) sayfa boyutu:

Offset = adres MOD 4096, VPN = adres DIV 4096

20000: VPN = 20000 / 4096 = 4, Offset = 20000 mod 4096 = 3616

32768: VPN = 32768 / 4096 = 8, Offset = 0 (tam sınırdadır)

60000: VPN = 60000 / 4096 = 14, Offset = 60000 mod 4096 = 3232

8 KB (8192 byte) sayfa boyutu:

Offset = adres MOD 8192, VPN = adres DIV 8192

20000: VPN = 20000 / 8192 = 2, Offset = 20000 mod 8192 = 3616

32768: VPN = 32768 / 8192 = 4, Offset = 0

60000: VPN = 60000 / 8192 = 7, Offset = 60000 mod 8192 = 2816

Soru 22 (İng): A computer whose processes have 1024 pages in their address spaces keeps its page tables in memory. The overhead required for reading a word from the page table is 5 nsec. To reduce this overhead, the computer has a TLB, which holds 32 (virtual page, physical page frame) pairs, and can do a lookup in 1 nsec. What hit rate is needed to reduce the mean overhead to 2 nsec?

Türkçe: Soru 22: Sayfa tablosu RAM'deyken bir kelime için ek maliyet 5 ns. TLB araması 1 ns. Ortalama maliyet 2 ns'ye düşürmek için gereken TLB hit oranı nedir?

Çözüm:

$$T_{\text{eff}} = h \times T_{\text{TLB}} + (1-h) \times T_{\text{PageTable}}$$

$$h \times 1 + (1-h) \times 5 = 2$$

$$h + 5 - 5h = 2$$

$$-4h = -3$$

$$h = 0.75$$

Cevap: TLB hit oranı en az %75 (0.75) olmalıdır.

Not: 32 girişlik TLB ile %75 hit oranı elde etmek,

1024 sayfalık adres uzayında oldukça mütevazı bir hedef.

Soru 35 (İng): A computer has four page frames. The time of loading, time of last access, and the R and M bits for each page are as shown below (times in clock ticks): Page 0: Loaded=126, Lastref=280, R=1, M=0 | Page 1: Loaded=230, Lastref=265, R=0, M=1 | Page 2: Loaded=140, Lastref=270, R=0, M=0 | Page 3: Loaded=110, Lastref=285, R=1, M=1. Which page will NRU, FIFO, LRU, Second Chance replace?

Türkçe: Soru 35: 4 frame, sayfaların yüklenme zamanı, son erişim zamanı ve R/M bitleri verili. NRU, FIFO, LRU, Second Chance'in hangi sayfayı seçeceğini bulunuz.

Çözüm:

Veriler:

Sayfa 0: Loaded=126, Lastref=280, R=1, M=0 → Sınıf 2 (R=1,M=0)

Sayfa 1: Loaded=230, Lastref=265, R=0, M=1 → Sınıf 1 (R=0,M=1)

Sayfa 2: Loaded=140, Lastref=270, R=0, M=0 → Sınıf 0 (R=0,M=0)

Sayfa 3: Loaded=110, Lastref=285, R=1, M=1 → Sınıf 3 (R=1,M=1)

NRU: En düşük sınıf = Sınıf 0 → Sayfa 2

FIFO: En eski yüklenme zamanı = 110 → Sayfa 3

LRU: En eski son erişim = 265 → Sayfa 1

Second Chance: En eski sayfa = Sayfa 3 (Loaded=110)

Sayfa 3: R=1 → R=0 yap, listeye son ekle.

Sonraki en eski: Sayfa 0 (Loaded=126), R=1 → R=0, son ekle.

Sonraki: Sayfa 2 (Loaded=140), R=0 → evict! → Sayfa 2

Soru 45 (İng): Explain the difference between internal fragmentation and external fragmentation. Which one occurs in paging systems? Which one occurs in systems using pure segmentation?

Türkçe: Soru 45: İç ve dış parçalanma arasındaki farkı açıklayın. Hangi tür sayfalama sistemlerinde, hangi tür saf segmentasyonda görülür?

Çözüm:

İç Parçalanma (Internal Fragmentation):

Tahsis edilen bellek bloğu isteğin boyutundan büyük olduğunda,

"atıl" kalan alan blok içinde kalır. Başka process bu alanı kullanamaz.

Örnek: 4 KB sayfa boyutu, process son sayfasının yalnızca 1 KB'ını kullanır

→ 3 KB iç fragmentation.

Dış Parçalanma (External Fragmentation):

Toplam boş bellek yeterli olsa da değişken boyutlu parçalar halinde

dağıldığından büyük bir istek karşılanamaz.

Örnek: 10 KB + 5 KB + 8 KB boş alan var ama 20 KB'lık bitişik alan yok.

Sayfalama: Yalnızca iç parçalanma (son sayfa tam dolmayabilir).

Sabit boyutlu frameler dış parçalanmayı ortadan kaldırır.

Saf Segmentasyon: Yalnızca dış parçalanma.

Segmentler değişken boyutlu olduğu için zamanla boş alanlar parçalanır;

compaction (sıkıştırma) gerekebilir.
